

A Survey on Querying Encrypted Data for Database as a Service

Rama Roshan Ravan
Advanced Informatics School
Universiti Teknologi Malaysia
Kuala Lumpur, Malaysia
rrrama2@live.utm.my

Norbik Bashah Idris
Advanced Informatics School
Universiti Teknologi Malaysia
Kuala Lumpur, Malaysia
norbik@ic.utm.my

Zahra Mehrabani
Asian Pacific University
UCTI University
Kuala Lumpur, Malaysia
haleh.mehrabani@gmail.com

Abstract— using database encryption to protect data in some situations where access control is not solely enough is inevitable. Database encryption provides an additional layer of protection to conventional access control techniques. It prevents unauthorized users, including intruders breaking into a network, from viewing the sensitive data. As a result data keeps protected even in the incident that database is successfully attacked or stolen. However, data encryption and decryption process result in database performance degradation. In the situation where all the information is stored in encrypted form, one cannot make the selection on the database content any more. Data should be decrypted first, so an unwilling tradeoff between the security and the performance is normally forced. The appropriate approaches to increase the performance are methods to deal directly with the encrypted data without firstly decrypting them. This paper while introduce some methods for searching on encrypted data, provides a comparison study between current methods in terms of performance and security.

Keywords; *Privacy, DbaaS, Database Security, Database Outsourcing, Encrypted Database, Query on Encrypted Data.*

I. INTRODUCTION

In modern business markets, customer's data is one of the most critical assets for organizations. Today's marketing database contains data that is vital for all aspects of the business such as marketing, sales, finance, customer service, and product development. While these data are essential for businesses, but most of them prefer to outsource the maintaining process of this critical asset. There are many reasonable factors to put the businesses in the way of outsourcing data to the external service providers. Take advantage of vendor skill and experience, shift the risk of technological complexity to the supplier, and free themselves up from other distractions to focus on their core competency marketing are some popular reasons in this new idea.

In the other side, today's network based data management services make it more important to detect and deter faulty behaviors. Actually, new data management services should provide assures services without any malicious activities. This is bolded particularly when clients leave data management with specialized service provider [1]. In fact, in era of cloud computing, all resources and applications are delivered as a service over the Internet [2].

In this solution, because of the data most important situation in organizational asset, data management service played an important role.

However, a new emerging option in this era is illustrated by Database as a Service (DbaaS) structure. Based on this paradigm, data owner manage the DBMS and answer to user's query directly, rather than traditional client-server architecture. One of the most serious obstacles to the prevalent use of DbaaS is related to security issues [3]. In the other hand, in this new paradigm data owner should be sure about security principles like confidentiality, integrity, authenticity, and privacy.

Within a DbaaS pattern data and application are stored together in the external server that takes full responsibility of their management. In this paradigm, however, service availability may increase because of using external server, but data owner control on the data reduces. In fact, in data outsourcing paradigm, some new security issues like privacy is appear.

In data outsourcing paradigm, users and data owners are supposed that the external server is trusted with maintain outsourced data trustworthy. In fact, in this structure server warranty the availability of outsourced data and data are available to the clients whenever they are requested. However, confidentiality and privacy of the database content is a problem that server is not trusted with it. While, outsourced data may be made of vital information, that data owner wants to protect them against unauthorized parties. Therefore, preventing external server from making unauthorized accesses to the outsourced data is an important requirement in DbaaS paradigm.

II. PRIVACY PROTECTION OF DBAAS

Different resources express varied definitions for privacy. In some of them, privacy supposes equivalent to confidentiality, while the others mentions some different between these two subjects. As a matter of fact, privacy is related to the individual's right that dictate what extent of personal data, when and how revealed to the others [4]. Also, privacy means to have authority (storage, collection, communication, manipulation, access, and constitution) over the data, that means privacy is different and broader concept than confidentiality. So, privacy prevention is build on

access control mechanism, but to ensure about privacy, some issues such as the purpose of using data should be in concern. In fact, in term of privacy the user just should use data for the purpose that authorized.

Databases management systems as a main element in data management play a crucial role on web interactions and daily operations of organizations. Today, with expanded use of programs and web services and also increasing of Internet attacks the risk of misuse of the database is more than before. Securing data stored in databases in both internal and external interactions is a basic need of organizations. Damage and misuse of the data not only affects one user or program, but may have detrimental effects to go the whole organization [5].

Backup and access control mechanisms are used to protect the database system. Backups are to protect against natural damages such as floods, earthquakes, fire, disk failures, while database access control mechanisms shall protect against unauthorized access. Various models have been proposed for access control in databases [6]. However, access control is possible only when the attacker try to penetrate the database through the main interface. Access control do not prevent against unusual data access; for example, when the media containing the databases stolen or when database manager wants to be able to access sensitive data, access control will not prevent disclosure of sensitive data. In such cases other methods must be used for security.

In this condition, data encryption method used for preventing the disclosure of information in the database. In fact, this approach is complementary methods of access control. Both of these methods should be used in databases to store and provide secure access to information in databases [7]. Disclosure of encrypted data, even when the server databases are at risk, is impossible. With encryption the data that stored on the disk, if the attacker can gain access to raw data stored on disk, not to take information stored in it. Also encryption enables database managers to do their administrative tasks without endangering user's data.

III. LITERATURE REVIEW

Using cryptography for privacy protection cause new problems. One of the problems is executing of users queries on encrypted data. Database management system can not querying encrypted databases as simple as raw plaintext database. Solving this problem is addressed in two ways [8]: the first solution is to decode the entire data when executing queries. Decoding of data in response to each query execution reduces speed and total system performance. Methods are proposed for reducing the number of decryptions and then increase the speed of query execution, but in some applications, data encryption done because of insufficient confidence to the database server. Thus, placing the decryption key in database server means neglecting the primary goal is encryption. In such applications, the server

is not able to decrypt the data. The second solution, presented in ways that could directly, without decoding, search on encrypted data. Applying such methods to reduce costs of encryption and decryption, and thus improve the efficiency.

In designing a method for searching on encrypted data, one of the important purposes that must be considered is to minimize the computation in the end user side (decoding key holder) and reducing the communication overhead. In fact, Methods should be designed in the way that a greater volume of operations performed on the server.

Individuals and institutions require methods to search on encrypted data with the same quality on decrypted one. This means that if possible; search records, which contain a particular value, with logarithmic degree of database size. In addition it is possible, the data includes encrypted and decrypted data. Decrypted data access performance should not be reduced because the encrypted data [6]. Provide such methods are not easy. Providing this property may be put at risk the security of information.

There are several methods for searching encrypted data without decrypting them. However, all these methods are not applicable in the encrypted database. Many of them are for searching a word in encrypted documents. Methods that provided for encrypted database should present needs of this environment in term of searching and operations.

Encrypted database search methods in terms of applicable data types can be divided into two categories [9]: methods and procedures applicable to the numerical data that and string data. Conditions that are performed on the data string, commonly applied to equality (=) and pattern matching (LIKE) while this conditions in numerical data are equality and range matching (<, >, =). So, It seems logical that different methods are presented for each of them.

IV. TAXONOMY OF CURRENT METHODS

Proposed methods for improving query performance on encrypted data can be divided into two groups in terms of confidence in the server. The first group is in which the server is trustful and the second one in the way that the server is completely mistrust.

The first group of techniques aimed at reducing the number of encoding and decoding that is required to perform per query, which does not need decoding of all data for each query. In the second approaches, the aim is direct handling of encrypted data without decrypting them. It is clear that the latter methods are applicable in the case of server reliability and their speed is more than the former group. However, in some applications, in addition to speed, reducing the memory overhead is necessary and the second group methods have more overhead than the first one.

A. Trustful Server Based Methods

The purpose of these methods is reducing cryptography operation in each query execution. There are some methods that are introduced for reducing memory usage and

encryption/decryption time in the state of trustful server that this article covers two of them, namely PPC and FCE.

- **Partition Plaintext Cipher Method:** In this method, to reduce the required memory size and operation, each memory page is divided into two sub-pages. In a half-page, values of sensitive fields and in the other, values of non-sensitive attributes for each record are stored. In addition, each record is divided into two sub-records and one of them saves critical parameters and the other contains non-sensitive parameters. At the end of each half-page, there is a table to point to the end of the sub-records. Every time a page is brought into memory, the contents of sensitive data half page are decrypted, and the search is done on the decrypted data. By this way, the amount of operations required for encoding and decoding and also the memory required to store the encrypted data are decreased [10].
- **Fast Comparison Encryption Method:** In this method, encoding is performed at the memory pages level based on a permutation function.

$$P(x) = ax^3 + bx^2 + cx + d \quad (1)$$

In fact, a permutation function is assigned to each memory page for encoding the page's contents in which a , b , c and d are different for each page and keep encrypted in the page header. Whenever a page prepares for reading, a , b , c and d is decoded and then P is calculated [11].

B. Mistrust Server Based Methods

Most of the proposed methods in this case, suppose that server is reliable in terms of data storage and data availability, but it is not reliable in confidentiality and privacy. In fact, they consider that the server is honest but curious. In the rest on this section some kinds of this type of methods will be discussed.

- **Practical Techniques for Searches on Encrypted Data:** Song and her colleagues, is a trapdoor encryption method [12]. In this method, each document divide into the words with 12 bit length and then each word encrypt separately. In fact, Song used two layers of encryption [13]. In the first layer, each word encodes by using a symmetric cryptographic algorithm that uses as a secret key. This causes the server to not know anything about the client's enquiry. Second layer of encryption uses a pseudo-random number generator and two pseudo-random functions. This layer causes the server to search for the client's enquiry by using the trapdoor. This method, for finding each word, needs to search all the content of all the documents and there is not a possibility of selecting length search. In fact, this method is only able to search words with n or coefficient of n bits length. In addition, this method provides a limited pattern of searching.

- **Secure Indexes:** Goh used index and trapdoor ideas for searching on the encrypted documents [14]. In this method, indexes are created based on the bloom filter method. In this method, an index is created for each document. Before construction of the index, a unique number D_{id} is assigned to each document. Then each word in the document is inserted in the corresponding index by using Bloom filters. By this way, it is possible to check the word belonging to the document.

In Goh's method there is no possibility of frequency attacks. Furthermore, the searching time of this method is $O(n)$ in which n is equal to the number of documents. This method may also have a false positive result. Finally, this method is only able to run in equity and there is no possibility to search for patterns.

- **Executing SQL Over Encrypted Data:** Hacigumus and colleagues introduced a method based on adding a secure index in each tuple [15][16][17]. This method with using a symmetric cryptography algorithm encrypts each tuple, separately. For each attribute where searches are performed on it, a property named index is added to the database. Index information should be in a form that the server could search in the right way, as well as the original data should be hidden.

In the Hacigumus method, querying on data is done in two steps: first, the server tries to respond as accurately as possible and sends the result back to the client and then the client decrypts the result and processes it. For this two steps query, client should divide query Q into two queries Q_s (works on index and stores on server) and Q_c (works on transmitted result from server in client side).

Hacigumus method has some drawbacks. As first drawback, if the data is encrypted in a wrong way in which each domain involves a value, there is the possibility of frequency analysis. Furthermore, this method is used for numerical data and applying it to string data that has a wide data range is not very good. In defining range for string data maybe a lot of data is placed in a range, which means a lot of additional data is sent to the respond of a query that resulting in a high computational and communication overhead. Also, in JOIN operation, a lot of wrong data is sent to the client.

Another drawback of Hacigumus method is the impossibility of aggregate functions such as *MAX*, and *COUNT* on encrypted numerical data. So, Hacigumus [19] extended this approach and by using privacy homomorphism function, provides some abilities such as addition and multiplication functions. Also, some methods are introduced for improving and reducing positive mistakes of the Hacigumus method.

- **Order Preserving Encryption for Numeric Data:**

This method is suitable for range querying on numerical data [19]. It supposes that preliminary distribution of data is A . But data are encrypted in a way that supports preliminary distribution of A' . In fact, they present an encryption scheme for numeric data that allows any comparison operation to be directly applied on encrypted data.

This scheme handles updates gracefully and new values can be added without requiring changes in the encryption of other values. It allows standard database indexes to be built over encrypted tables and can easily be integrated with existing database systems. The proposed scheme had been design to be deployed in application environments in which the intruder can get access to the encrypted database, but does not have prior domain information such as the distribution of values and cannot encrypt or decrypt arbitrary values of his choice. The encryption is robust against estimation of the true value in such environments. Moreover, in this method, it is expected that data order is confidential, although revealing of order is a huge and important issue.

- **Balancing Confidentially and Efficiency in Untrusted Relational DBMSs:** Damiani proposed an index-based method [20]. In fact, this method is the same as the Hacigumus method [17]. For creating the index, data are classified by a collusion hash function. The advantage of this method rather than Hacigumus method is that it does not classify data sequentially and in this way, it increases the security.

The disadvantages of this type of classification are that there is no possibility of range searching on the encrypted data while it is possible in the Hacigumus method. For covering this weakness, Damiani used the encoded B-tree method. This technique allows the execution of range queries in Damiani method. But for each query it needs to perform many queries that equal to height of the tree, which is stored in the server.

- **Fast Query in Encrypted Character Data in Database:** Wang proposed an index-based method for querying encrypted string data [21][22]. In this method, for each string that has n character, an index is assigned, which has m bit length. Then, the string is extended. And by using a hash function a value, which is between 0 and m , is generated for each substring.

Wang approach can quickly execute SQL query on the encrypted data. For character data, it not only encrypts them, but also turns the character data into characteristic values via a characteristic function and stores them as additional fields. Same for numerical data, it not only encrypts them, but also creates its B+ tree index before the encryption in order to keep

the ordering of each record in the index. Furthermore, it gives the algorithms of querying the encrypted data based on the storage models.

By using Wang method, arbitrary patterns queries on encrypted data are possible. For searching patterns, the string value contained in the pattern is extended and the values are allocated to each two successive letters obtained. These values indicate the bits of index, which should be 1 to satisfy the query condition. After that, these values are sent to the server and it checks the bit values of index. If all these values are assigned to 1, the value of index is sent to the client as a response.

In this Method, if the m is large, there is a possibility of breaking the frequency analysis and also for the small m , large numbers of false data will be sent to the client.

- **Privacy Preserving Queries on Encrypted Data:**

Yang proposed a method [23] for querying encrypted database that is same as Song method [13]. This method uses encrypted attribute with T' that is the output of $E(f(T), T) = T_2$ for storing each attribute in the database in which E is an encryption function and T is the value of attribute.

When searching T' in the database, client should pass the $f(T')$ to the server. Then, each encrypted value (T) that satisfies $E(f(T), T) = T_2$ is sent to the client as an answer.

One of the advantages obtain with the Yang method is that for searching a word, there is no need for sending the word to the server in which it enhances the security of the method. But storing the encrypted value of each attribute can lead to frequency analysis. Like other methods, this method is only able to make a definitive match query.

V. METHODS COMPARISON

In this section, various types of methods that have been introduced in this article are compared in terms of query type, and time requirement. The following table shows this comparison in which solid circles mean full support, not quite filling circles are meant as to support partially and empty circles are meant as a lack of support for query.

TABLE I. METHODS COMPARISON

Method	Query Type				Searching Time	Security	
	Equality	Pattern	Range	Aggregation Functions		Frequency Analysis	Chosen Plaintext Attack
Executing SQL Over Encrypted Data	●	○	●	○	$O(\log n)$	×	×
Practical Techniques for Searches on	●	●	○	○	$O(n)$	×	✓

Method	Query Type				Searching Time	Security	
	Equality	Pattern	Range	Aggregation Functions		Frequency Analysis	Chosen Plaintext Attack
Encrypted Data							
Secure Indexes	●	●	○	○	$O(n)$	✓	✓
Fast Query in Encrypted Character Data in Database)	●	●	○	○	$O(n)$	×	×
Privacy Preserving Queries on Encrypted Data	●	○	○	○	$O(n)$	✓	✓
Balancing Confidentially and Efficiency in Untrusted Relational DBMSs	●	○	●	○	$O(\log n)$	✓	×
Efficient execution of aggregation queries over encrypted relational databases	●	○	○	●	$O(\log n)$	×	×
Order Preserving Encryption for Numeric Data	●	○	●	○	$O(\log n)$	✓	×

VI. CONCLUSION

In this chapter, a variety of search methods in encrypted databases and documents are presented and some examples are reviewed and evaluated. Researches show that most current methods just provide equality query type on encrypted string data, and a safe method, which can search arbitrary patterns (LIKE), has not been introduced yet.

The next important thing that is seen in the current methods is the number of records, which is retrieved to respond to client's query in which it is often more than the records that have been requested. Therefore, after decrypting data in the client side, there is a need to perform queries in decrypted values for finding the results in which it forces computational overhead to the client, and also, the client should be able to work with relational values. Another problem is that most existing methods are vulnerable to the frequency attack and the methods that are resistant against these attacks have a search speed based on database record number.

REFERENCES

- [1] Bertino, E. and R. Sandhu (2005). "Database security-concepts, approaches, and challenges." Dependable and Secure Computing, IEEE Transactions on 2(1): 2-19
- [2] Heiser, J. and M. Nicolett (2008). "Assessing the security risks of cloud computing." *Gartner Report*.
- [3] Ercan, T. (2010). "Effective use of cloud computing in educational institutions." *Procedia-Social and Behavioral Sciences* 2(2): 938-942
- [4] P. C. K. Hung and V. S. Y. Cheng. (2009) Privacy. In L. Liu, M.T. Ozu editors, *Encyclopedia of Database Systems (EDS)*, Springer.
- [5] He, J. and M. Wang (2001). *Cryptography and relational database management systems*, IEEE.
- [6] Sion, R. (2008). Towards secure data outsourcing. *Handbook of Database Security*, Springer: 137-161.
- [7] O'Flaherty, K. W., Stellwagen Jr, R. G., Walter, T. A., Watts, R. M., Ramsey, D. A., Veldhuisen, A. W., et al. (2001). Privacy-enhanced database.
- [8] De Capitani Di Vimercati, S., Foresti, S., and Samarati, P. (2012). Protecting data in outsourcing scenarios. *Handbook on securing cyber-physical critical infrastructure*.
- [9] Elmasri, R. (2008). *Fundamentals of database systems*. Pearson Education India.
- [10] Iyer, B., Mehrotra, S., Mykletun, E., Tsudik, G., and Wu, Y. (2004). A framework for efficient storage security in RDBMS. *Advances in Database Technology-EDBT 2004*, 627-628.
- [11] Ge, T., and Zdonik, S. (2007). Fast, secure encryption for indexing in a column-oriented dbms 676-685.
- [12] Song, D. X., Wagner, D., and Perrig, A. (2000). Practical techniques for searches on encrypted data 44-55.
- [13] Brinkman, R., Feng, L., Etalle, S., Hartel, P. H., and Jonker, W. (2003). Experimenting with linear search in encrypted data.
- [14] Goh, E. J. (2003). Secure indexes. An early version of this paper first appeared on the Cryptology ePrint Archive on October 7th.
- [15] Hacigümüş, H., Iyer, B., Li, C., and Mehrotra, S. (2002). Executing SQL over encrypted data in the database-service-provider model 216-227.
- [16] Hacigumus, H., Iyer, B., and Mehrotra, S. (2002a). Providing database as a service 29-38.
- [17] Hacigümüş, H., Iyer, B., and Mehrotra, S. (2005). Query optimization in encrypted database systems. *Lecture notes in computer science*, 43-55.
- [18] Hacigümüş, H., Iyer, B., and Mehrotra, S. (2004). Efficient execution of aggregation queries over encrypted relational databases 633-650.
- [19] Agrawal, R., Kiernan, J., Srikant, R., and Xu, Y. (2004). Order preserving encryption for numeric data 563-574.
- [20] Damiani, E., Vimercati, S., Jajodia, S., Paraboschi, S., and Samarati, P. (2003). Balancing confidentiality and efficiency in untrusted relational DBMSs 93-102.
- [21] Wang, Z. F., Dai, J., Wang, W., and Shi, B. L. (2005). Fast query over encrypted character data in database. *Computational and Information Science*, 1027-1033.
- [22] Wang, Z. F., Wang, W., and Shi, B. L. (2005). Storage and query over encrypted character and numerical data in database 77-81.
- [23] Yang, Z., Zhong, S., and Wright, R. (2006). Privacy-preserving queries on encrypted data. *Computer Security-ESORICS 2006*, 479-495.